TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 2

Remote dev (WSL/SSH), Module 00 checkpoint, first terminal drills

Phase 1 — Foundations · Weeks 1–6 · Modules 00–02

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 00.05 Connecting to WSL
2. 00.06 Connecting over SSH
3. 00.07 Final Checkpoint
4. Module 01 — Ubuntu Terminal Foundations: overview
5. 01.00 Setup Check
6. 01.01 Where Am I?

Learning objective: Connect VS Code to WSL using the WSL extension, open a folder in the Linux home directory, and verify the terminal is Linux by running `uname -a`.

Lesson 05: Connecting to WSL

Time: ~20 min

> **This lesson is for Windows students only.** Mac and Linux students: you already have a real Unix terminal — skip ahead to lesson 06, or pair up with a Windows neighbor and help them.

What is WSL?

WSL stands for **Windows Subsystem for Linux**. It is a real Linux operating system that runs *inside* Windows, side by side with your normal Windows environment. You don't dual-boot, you don't run a VM in the old slow way — you just open WSL and you're in a Linux shell.

Why do we use WSL in this course? Because the rest of the course (Module 01 onward) assumes you have a Linux terminal. On Windows, the cleanest way to get one is WSL.

Step 1: check whether WSL is installed

1. Open Windows **PowerShell** (press the Windows key, type "powershell", press Enter).
2. In PowerShell, type:

``` `wsl --status` ```

1. Two outcomes:

- **It prints version info** like "Default Distribution: Ubuntu" — WSL is installed. Skip to Step 3.
- **It prints "WSL is not installed"** or an error — go to Step 2.

Step 2: install WSL (if needed)

> Heads up: this requires a **reboot**. Your instructor will tell you whether to do this in class or at home.

1. In **PowerShell as administrator** (right-click PowerShell → "Run as administrator"), run:

``` `wsl --install` ```

1. Wait for it to finish. It will install WSL and a default Ubuntu distribution.

2. **Reboot your computer.**
3. After reboot, Ubuntu will launch automatically and ask you to **create a Linux username and password.**

- The password **does not show on screen** when you type — no asterisks, no dots, nothing. Just type it carefully and press Enter. Then confirm it.
- Pick a simple lowercase username. It does **not** have to match your Windows username.
- Remember this password. You'll need it for `sudo` commands in Module 01.

1. Once you see the Ubuntu prompt (`yourname@hostname:~$`), WSL is installed.

Step 3: install the WSL extension in VS Code

Remote support isn't built into VS Code — it comes from an extension you install yourself.

1. Open VS Code.
2. Open the extensions side bar (`Ctrl+Shift+X`).
3. Search for **WSL**.
4. The one you want is published by **Microsoft**, full id `ms-vscode-remote.remote-wsl`.
5. Click **Install**.

Step 4: connect VS Code to WSL

1. Look at the **bottom-left corner** of the VS Code window. There's a small green or blue button — it might just be two angle brackets `><`, or it might say something like "Open a Remote Window."
2. Click it.
3. A menu drops down from the top of the window. Choose **"Connect to WSL"** (or "WSL: Connect to WSL").
4. VS Code reloads. The bottom-left corner now shows **"WSL: Ubuntu"** (or whichever distro you have) on a green background.

You are now running VS Code's "remote" half inside WSL. Your editor is still on Windows; the files, terminal, and tools are all in Linux.

Step 5: open a Linux folder

1. **File → Open Folder…**
2. The path bar should now show Linux paths. Navigate to `/home/<your-linux-username>/`.
3. Click **OK**.
4. The file explorer in the side bar fills with whatever is in your Linux home directory (probably empty or just a couple of dotfiles).

Step 6: confirm the terminal is Linux

1. Open the integrated terminal (`` `Ctrl+` ``).
2. The prompt should look like:

``yourname@hostname:~$`` ``That's a Linux prompt. Not PS C:\>`` — that would be PowerShell.

1. Type:

``uname -a`` The output should mention "Linux" somewhere. If it does, you really are inside WSL.

Checkpoint

You're done when:

- [ ] `wsl --status` in PowerShell shows WSL is installed.
- [ ] You installed the **WSL** extension in VS Code (publisher Microsoft).
- [ ] The bottom-left of your VS Code window shows **"WSL: Ubuntu"** (or your distro name) on green.
- [ ] You opened a folder under `/home/<your-linux-username>/`.
- [ ] You opened the integrated terminal and ran `uname -a` and saw output containing "Linux."

If anything fails, raise your hand. WSL has more failure modes than the rest of the course combined and they're better to fix in person.

Lab

Objective: Prove you have a working Unix shell — via WSL on Windows, or your native terminal on Mac/Linux.

Task: Windows students: work through this lesson — install WSL if needed, install the WSL extension, connect VS Code to WSL, and run `uname -a` in the integrated terminal. Mac/Linux students: open your native terminal (Terminal app on Mac) and run `uname -a` — that's your whole lab.

Submit: Windows — one screenshot of your VS Code window showing the green **"WSL: Ubuntu"** indicator in the bottom-left and the output of `uname -a` in the integrated terminal. Mac/Linux — one screenshot of `uname -a` and its output in your native terminal.

Pass criteria:

Windows:

- [ ] The bottom-left indicator shows "WSL: Ubuntu" (or your distro name) on green.
- [ ] The `uname -a` output is visible and mentions "Linux."

Mac/Linux:

- [ ] Screenshot shows your native terminal with `uname -a` and its output (mentioning "Darwin" on Mac, or "Linux").

Unlocks: 06-ssh-remote.md

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Connect VS Code to an instructor-provided remote host with the Remote - SSH extension, open a remote folder, and confirm the session by running `hostname` and `whoami`.

Lesson 06: Connecting over SSH

Time: ~20 min

What is SSH?

SSH stands for **Secure Shell**. It is the standard way to connect to and control a remote computer over the network. You sit at your laptop, run an SSH command, type a password (or use a key), and you get a terminal on the remote computer — as if you were sitting in front of it.

The VS Code **Remote - SSH** extension takes this one step further: not only do you get a remote terminal, you also get the editor working on the remote files. You edit a file in VS Code, the file lives on the remote, and you can run it in a remote terminal — all without leaving your local VS Code window.

Before you start

Your instructor will give you the exact connection details for today:

- **Host:** `<INSTRUCTOR-PROVIDED>` — e.g. `ec2-12-34-56-78.compute.amazonaws.com`
- **User:** `<INSTRUCTOR-PROVIDED>` — e.g. `student`
- **Auth:** either a **password** or an **SSH key file** the instructor sent you.

Write these down before you go further.

Step 1: install the Remote - SSH extension

Remote support isn't built into VS Code — it comes from an extension you install yourself.

1. Open the extensions side bar (`Ctrl+Shift+X`).
2. Search for **Remote - SSH**.
3. Publisher: **Microsoft**, id `ms-vscode-remote.remote-ssh`.
4. Click **Install**.

Step 2 (key auth only): place your key file

If your instructor gave you a **password**, skip this step.

If they gave you a **key file** (a small file usually named like `student_key` or `id_ed25519` with no extension):

1. Save the file to:

- **Windows:** `C:\Users\<You>\.ssh\` — create the `.ssh` folder if it doesn't exist.
- **Mac/Linux:** `~/.ssh/`

1. **Mac/Linux only:** in a terminal, run:

``` `chmod 600 ~/.ssh/<your-key-filename>` ``` This locks down the file's permissions so SSH will accept it. SSH refuses keys that anyone else can read.

1. **Windows:** if Windows complains about permissions later, right-click the key file → Properties → Security → Advanced → "Disable inheritance" → "Remove all" → then add yourself with Read access only.

Step 3: connect

1. In VS Code, click the **bottom-left corner** button (the `><` angle brackets) — same place as the WSL button.
2. From the dropdown choose **"Connect to Host…"** (or "Remote-SSH: Connect to Host…").
3. Choose **"Add New SSH Host…"**.
4. Type the SSH command exactly as your instructor gave it. For password auth:

``` `ssh student@ec2-12-34-56-78.compute.amazonaws.com` ``` `For key auth:` ``` `ssh -i ~/.ssh/student_key student@ec2-12-34-56-78.compute.amazonaws.com` ``` (Substitute the real values your instructor gave you.)

1. Press Enter.
2. VS Code asks where to save the SSH config. Pick the first option (your user `~/.ssh/config`).
3. Click the bottom-left button again → "Connect to Host…" → your new host should be in the list. Click it.

Step 4: trust and authenticate

The first time you connect to a new host, two things happen:

1. A prompt appears: **"The authenticity of host ... can't be established. Are you sure you want to continue connecting (yes/no/[fingerprint])?"** Type **`yes`** and press Enter. (Pressing Enter alone without typing yes does nothing — the question doesn't have a default.)
2. If you used password auth, a password prompt appears. Type the password (it won't show on screen) and press Enter.

VS Code's bottom-left now reads **"SSH: <hostname>"** on green. You are connected.

Step 5: open a folder on the remote

1. **File → Open Folder…**
2. The path bar shows paths on the **remote** machine, not your laptop. Navigate to your home directory there (something like `/home/student/`).
3. Click **OK**.

Step 6: confirm the terminal is on the remote

1. Open the integrated terminal (`Ctrl+`).
2. The prompt should look like:

`student@ec2-12-34-56-78:~$` That hostname is the **remote** machine, not your laptop.

1. Run:

`hostname` The output should be the remote's hostname.

1. Run:

`whoami` The output should be the username your instructor gave you (e.g. student).

If both are correct: every command you run in this terminal runs on the remote. Every file you save in this VS Code window saves to the remote. Your laptop is now a thin client.

Disconnecting

Click the bottom-left **"SSH: ..."** button → **"Close Remote Connection"**. VS Code reloads back to your local environment. The remote keeps running; you just stopped talking to it.

Checkpoint

You're done when:

- [ ] You installed the **Remote - SSH** extension by **Microsoft**.
- [ ] You added the instructor-provided host as an SSH target.
- [ ] The bottom-left of your VS Code window shows **"SSH: <hostname>"** on green.
- [ ] You opened a folder on the remote machine.
- [ ] You ran `hostname` and `whoami` in the integrated terminal and the output matches the remote.

If you get a `Permission denied (publickey)` error, your key isn't being used or its permissions are wrong — flag the instructor.

Lab

Objective: Connect VS Code to a remote machine over SSH and prove your terminal is running there.

Task: Work through this lesson against the college's shared server — host `<INSTRUCTOR-PROVIDED>`, user `<INSTRUCTOR-PROVIDED>` — install the Remote - SSH extension, connect, open a remote folder, and run `hostname` and `whoami` in the integrated terminal.

Submit: One screenshot of your VS Code window showing the green **"SSH: <hostname>"** indicator in the bottom-left and the output of `hostname` and `whoami` in the integrated terminal.

Pass criteria:

- [ ] The bottom-left indicator shows "SSH: <hostname>" on green.
- [ ] The `hostname` output matches the instructor-provided host, not your laptop.
- [ ] The `whoami` output is the instructor-provided username.

Unlocks: 07-final-checkpoint.md

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Demonstrate unaided use of VS Code by toggling the terminal from the command palette, installing an extension, connecting to WSL or SSH, and capturing a screenshot showing the green remote indicator with `whoami` and `pwd` output.

Lesson 07: Final Checkpoint

Time: ~15 min — **no instructor help**.

This is a self-test. Do every step in order, on your own. If you get stuck, **re-read the relevant lesson** — don't ask the instructor or your neighbor. The point is to prove to yourself that you can drive VS Code without a guide.

The test

1. **Command palette + terminal.** Open the command palette with the keyboard shortcut. Use it to toggle the integrated terminal (no clicking the menu, no Ctrl+backtick — only the command palette).
2. **Install an extension.** From the extensions side bar, install **"Markdown All in One"** by **Yu Zhang**. Verify it appears under the **INSTALLED** section.
3. **Connect to a remote environment.** Either:

- Connect to **WSL** (Windows students), **or**
- Connect to **SSH** using the instructor-provided host (any student). Confirm the bottom-left of your VS Code window shows **"WSL: ..."** or **"SSH: ..."** on green.

1. **Open a remote folder.** With the connection active, open any folder on the remote/WSL side. Your file explorer should show its contents.
2. **Run two commands in the remote terminal.** Open the integrated terminal — it should be a remote terminal (its prompt is the remote/WSL hostname). Run:

``` `whoami pwd` ``` Both should print results from the remote, not your laptop.

Lab

This is the Module 00 phase-gate lab.

Objective: Prove you can drive VS Code — command palette, extensions, remote connection, remote terminal — without a guide.

Task: Complete all five steps of the test above on your own: toggle the terminal from the command palette, install "Markdown All in One" by Yu Zhang, connect to **WSL** (Windows students) **or SSH** using the instructor-provided host (any student), open a remote folder, and run `whoami` and `pwd` in the remote terminal.

Submit: One screenshot of your VS Code window that shows, at the same time: the bottom-left status bar with **"WSL: ..."** or **"SSH: ..."** on green, and the integrated terminal with the output of `whoami` and `pwd` visible. Submit it in D2L.
```

Pass criteria:

- [ ] The green WSL or SSH indicator is visible in the bottom-left.
- [ ] The output of both `whoami` and `pwd` is visible in the terminal.
- [ ] Everything is in a single screenshot.

If yours is missing one of these, fix it before submitting. If you really can't get there in 15 minutes, raise your hand at the end of the time block — we'll regroup.

Unlocks: Module 01 — Ubuntu Terminal Foundations (phase gate)

What's next

Module 01 (`../01-ubuntu-terminal/`) starts with you sitting in front of a working terminal — exactly where this checkpoint leaves you. Two and a half hours of typing drills are about to make all those terminal commands second nature. Get a glass of water and stretch your fingers.

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Module 01 — Ubuntu Terminal Foundations

What you'll be able to do after this module

- Navigate the Linux filesystem confidently using `pwd`, `ls`, and `cd`
- Create, copy, move, rename, and delete files and directories from the terminal
- Read and edit files from the terminal using `cat`, `head`, `tail`, `less`, `nano`, and output redirection
- Update an Ubuntu system and install packages with `apt` using `sudo`
- Interpret and modify file permissions and ownership with `chmod` and `chown`
- Diagnose basic network connectivity and DNS with `ping`, `curl`, `dig`, and `ss`
- Inspect running processes and service status with `ps`, `top`, `systemctl`, and `journalctl`
- Chain commands with pipes and `grep`, and manage users and groups in preparation for the docker group

Lessons and objectives

#	Lesson	Learning objective
00	Setup Check	Verify that the Ubuntu environment is course-ready by checking the OS release, internet connectivity, DNS resolution, and sudo access.
01	Where Am I?	Navigate the filesystem without hesitation using `pwd`, `ls`, `cd` with absolute paths, `cd -`, and the `~` home shortcut.
02	Making Things	Create the ~/docker-course working directory and practice creating, copying, moving, renaming, and deleting files and directories with `mkdir`, `touch`, `cp`, `mv`, and `rm`.
03	Looking Inside Files	Create, read, and edit files from the terminal using `echo` with `>` and `>>` redirection, `cat`, `head`, `tail`, `less`, `wc -l`, and `nano`.
04	Sudo and System Updates	Perform a full system update with `sudo apt update`, `sudo apt upgrade -y`, and `sudo apt autoremove -y`.
05	Installing Software	Install and verify the Docker prerequisite packages (curl, wget, git, htop, tree, ca-certificates, gnupg) using `sudo apt install`.
06	Permissions Drills	Read permission strings and change file permissions and ownership using `chmod` in symbolic and numeric forms and `chown` with sudo.
07	Networking Drills	Diagnose network readiness for Docker Hub using `ip a`, `ping`, `curl`, `ss -tuln`, and `dig`.
08	Processes and Services	Inspect running processes and service status using `ps aux`, `top`, `htop`, `systemctl`, `journalctl`, `uptime`, `free -h`, and `df -h`.
09	Pipes and Grep	Chain commands together with pipes and search files using `grep` (including -v, -i, and -r), `wc -l`, `cut`, `sort`, and `uniq`.
10	User and Group Setup	Inspect user and group membership with `id` and `groups`, add a user to a group with `sudo usermod -aG`, and refresh group membership with `newgrp`.
11	Final Checkpoint	Demonstrate every skill from the module by running the cumulative command drill from memory to prove the environment is ready for Docker installation.

Each lesson is one learning object with a paired lab. Work in order — completing a lab unlocks the next lesson. The module ends with a **Check on Learning** quiz and a hands-on final checkpoint.

Learning objective: Verify that the Ubuntu environment is course-ready by checking the OS release, internet connectivity, DNS resolution, and sudo access.

Lesson 00: Setup Check

By the end, you will have confirmed that your Ubuntu environment is ready for this course.

Estimated time: ~5 minutes of typing

---

1. Check your Ubuntu version — this confirms you're on a supported release.

```
lsb_release -a
```

Expected output (version may differ):

```
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.x LTS
Release:        22.04
Codename:       jammy
```

(You should see 22.04 or 24.04. Either is fine.)

2. Confirm you have internet access — every later step needs this.

```
ping -c 3 8.8.8.8
```

Expected output:

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=12.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=11.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=12.1 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
```

(Times will differ. 0% packet loss is what matters.)

3. Confirm DNS resolution works — we'll need to download packages by name.

```
ping -c 3 google.com
```

Expected output:

```
PING google.com (142.250.x.x) 56(84) bytes of data.
64 bytes from ...: icmp_seq=1 ttl=117 time=12.3 ms
...
3 packets transmitted, 3 received, 0% packet loss
```

(The IP will vary. 0% packet loss is what matters.)

4. Confirm your user has sudo access — you'll need this for installing packages.

```
sudo whoami
```

Expected output:

```
root
```

> **Warning:** Be careful with `sudo`. It runs commands as the root user, which has unrestricted access to the entire system. A typo in a `sudo` command can delete files, break your OS, or cause data loss. Always double-check before pressing Enter.

(You may be prompted for your password. Type it — characters won't appear on screen.)

5. Check your username — we'll reference this throughout the course.

```
whoami
```

Expected output:

```
your-username
```

(This should NOT say "root". If it does, you're logged in as root — create a normal user first.)

6. Run one more pwd to see where you start.

```
pwd
```

Expected output:

```
/home/your-username
```

Checkpoint

Run each command and confirm the expected result:

```
lsb_release -r
```

You should see `Release: 22.04` or `Release: 24.04`.

```
ping -c 1 8.8.8.8
```

You should see `1 packets transmitted, 1 received, 0% packet loss`.

```
sudo whoami
```

You should see `root`.

Lab

Objective: Confirm your Ubuntu environment is ready for this module — supported release, working network, and sudo access.

Task: Work through the six drills in this lesson, then run the three Checkpoint commands (`lsb_release -r`, `ping -c 1 8.8.8.8`, `sudo whoami`) in one terminal.

Submit: One screenshot (or pasted transcript) of the three Checkpoint commands and their output: `lsb_release -r`, `ping -c 1 8.8.8.8`, and `sudo whoami`.

Pass criteria:

- [ ] `lsb_release -r` shows `Release: 22.04` or `Release: 24.04`. (Fresh cloud VM said "command not found"? Installing it first with `sudo apt install -y lsb-release` is fine.)
- [ ] `ping -c 1 8.8.8.8` shows `0% packet loss`.
- [ ] `sudo whoami` prints `root`.
- [ ] If `ping google.com` failed on WSL, you applied the documented DNS fix (`nameserver 8.8.8.8` in `/etc/resolv.conf`) and names now resolve.

Unlocks: 01-where-am-i.md

Next up: 01-where-am-i.md — Navigate the filesystem

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Navigate the filesystem without hesitation using `pwd`, `ls`, `cd` with absolute paths, `cd -`, and the `~` home shortcut.

Lesson 01: Where Am I?

By the end, you will navigate the filesystem without hesitation.

Estimated time: ~15 minutes of typing

---

1. Print your current directory — this is the command you'll type most often.

```
pwd
```

```
/home/your-username
```

2. Confirm who you're logged in as.

```
whoami
```

```
your-username
```

3. Check this machine's hostname.

```
hostname
```

```
your-hostname
```

(Output varies — just confirm it returns something.)

4. List what's in your home directory.

```
ls
```

```
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

(Your output may differ — a fresh install may show nothing or different folders.)

5. List again with details — this shows permissions, owner, size, and date.

```
ls -la
```

```
total 32
drwxr-x--- 4 your-user your-user 4096 Jan  1 12:00 .
drwxr-xr-x 3 root      root      4096 Jan  1 12:00 ..
-rw-r--r-- 1 your-user your-user  220 Jan  1 12:00 .bash_logout
-rw-r--r-- 1 your-user your-user 3771 Jan  1 12:00 .bashrc
-rw-r--r-- 1 your-user your-user  807 Jan  1 12:00 .profile
```

(Output will vary. Notice the `.` files — those are hidden files.)

6. Move to the root of the filesystem.

```
cd /
```

(No output — that's normal. cd is silent on success.)

7. Confirm you moved.

```
pwd
```

```
/
```

8. List what's at the root.

```
ls
```

```
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys
tmp  usr  var
```

9. Look at the details of root-level directories.

```
ls -la
```

```
total 64
drwxr-xr-x  19 root root 4096 Jan  1 12:00 .
drwxr-xr-x  19 root root 4096 Jan  1 12:00 ..
drwxr-xr-x   2 root root 4096 Jan  1 12:00 bin
drwxr-xr-x   3 root root 4096 Jan  1 12:00 boot
...
```

10. Move into /etc — this is where system config files live.

```
cd /etc
```

11. Confirm.

```
pwd
```

```
/etc
```

12. List this directory.

```
ls
```

(You'll see a long list of config files and folders.)

13. Go back to where you just were using cd -.

```
cd -
```

```
/
```

(cd - takes you to your PREVIOUS directory. It also prints where it took you.)

14. Confirm.

```
pwd
```

```
/
```

15. Use cd - again to bounce back to /etc.

```
cd -
```

```
/etc
```

16. Confirm.

```
pwd
```

```
/etc
```

17. Go to your home directory using cd with no arguments.

```
cd
```

(No output.)

18. Confirm you're home.

```
pwd
```

```
/home/your-username
```

19. Go to /var/log — this is where system logs live.

```
cd /var/log
```

20. Confirm and list.

```
pwd
```

```
/var/log
```

```
ls
```

(You'll see log files like syslog, auth.log, kern.log, etc.)

21. Go home using the tilde shortcut.

```
cd ~
```

(No output.)

22. Confirm.

```
pwd
```

```
/home/your-username
```

23. Move to /tmp — a directory anyone can write to.

```
cd /tmp
```

24. Confirm and list.

```
pwd
```

```
/tmp
```

```
ls -la
```

(Contents will vary.)

25. Jump home one more time — drill it.

```
cd ~
```

26. Confirm.

```
pwd
```

```
/home/your-username
```

27. One final drill — move to /usr/bin where programs live.

```
cd /usr/bin
```

28. Confirm and list a few items.

```
pwd
```

```
/usr/bin
```

```
ls | head
```

(You'll see the first 10 programs installed on your system.)

29. Go home.

```
cd
```

30. Confirm.

```
pwd
```

```
/home/your-username
```

Checkpoint

Run each command and confirm the result:

```
pwd
```

You should be in `/home/your-username`.

```
cd /etc && pwd
```

You should see `/etc`.

```
cd - && pwd
```

You should be back in `/home/your-username`.

```
cd / && ls
```

You should see the root directories (bin, boot, dev, etc...).

```
cd ~ && pwd
```

You should be in `/home/your-username`.

Lab

Objective: Navigate the filesystem with `pwd`, `cd`, and `ls` — including `cd -`, `cd ~`, and bare `cd` — without hesitation.

Task: Work through the 30 navigation drills in this lesson, then run the five Checkpoint commands in order in one terminal.

Submit: One screenshot (or pasted transcript) of the five Checkpoint commands and their output: `pwd`, `cd /etc && pwd`, `cd - && pwd`, `cd / && ls`, `cd ~ && pwd`.

Pass criteria:

- [ ] The first `pwd` prints `/home/<your-username>`.
- [ ] `cd /etc && pwd` prints `/etc`.
- [ ] `cd - && pwd` prints your home directory again.
- [ ] `cd / && ls` lists the root directories (`bin`, `boot`, `etc`, `home`, ...).
- [ ] `cd ~ && pwd` prints `/home/<your-username>`.

Unlocks: 02-making-things.md

Next up: 02-making-things.md — Create files and directories

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.